



# 力扣学习

## Elegant $\text{\LaTeX}$ 经典之作

作者: Ethan Deng & Liam Huang

组织: Elegant $\text{\LaTeX}$  Program

时间: April 9, 2022

版本: 4.3

自定义: 信息



不要以为抹消过去，重新来过，即可发生什么改变。——比企谷八幡

# 目录

|                           |           |
|---------------------------|-----------|
| <b>第 1 章 数据库基础语法</b>      | <b>1</b>  |
| 1.1 计算字符 . . . . .        | 2         |
| 1.2 函数 . . . . .          | 4         |
| 1.3 联结表 13/22 . . . . .   | 8         |
| 1.4 union . . . . .       | 10        |
| 1.5 高级用法 . . . . .        | 12        |
| <b>第 2 章 练习</b>           | <b>14</b> |
| 2.1 基础 . . . . .          | 14        |
| 2.2 连接 . . . . .          | 14        |
| 2.3 函数 . . . . .          | 16        |
| 2.4 聚合 . . . . .          | 17        |
| 2.5 高级查询和链接 . . . . .     | 21        |
| 2.5.1 窗口函数 . . . . .      | 22        |
| 2.5.2 union all . . . . . | 24        |
| 2.6 子查询 . . . . .         | 25        |

# 第 1 章 数据库基础语法

SQL 是 Structured Query Language 的缩写，中文翻译为“结构化查询语言”

## 定义 1.1 (SELECT 语句)

在 SQL 中，**SELECT** 语句是用于从数据库表中检索数据的核心指令。其基本语法结构由以下两个核心部分组成：

- 选什么 (**What to select**)：紧跟在 **SELECT** 关键字之后，用于指定需要查询的列名（字段）。
- 从哪里选 (**From where to select**)：紧跟在 **FROM** 关键字之后，用于指定数据所在的表名。

## 东邪的 tip 1.1

如果要选择多个关键字每一个关键字都要用，隔开。如果需要选择表中的所有列，可以使用通配符 **\***。

## 东邪的 tip 1.2

如果要选择名字不重复的就用 **distinct** **distinct** 是形容词所以放在要选什么的前面。**distinct** 对他后面的都有效而不是只是对他后面一个位置的有效。必须紧紧跟着 **select**

## 东邪的 tip 1.3

不同的系统选择前五行语法不一样，甲骨文的是 **where rownum** 小于等于 5。如果想从要 6 到 10 行就 **limit 5 offset 5** **offset** 是偏离的意思也就偏离 5 的定语是从 1 偏离 5 也就是从第 6 个开始，他前面的就是还需要多少行。1 到 10 有 10 个数，1 到 5 有五个数所以 5-1 需要再加 1

 **笔记** 1. 物理位置 vs. 逻辑索引在 SQL 的底层逻辑里，它不把第一行看作“1”，而是看作距离起点的偏移量 (**Offset**) 为 0 的位置。2. 为什么你的“偏离 5”等于“第 6 行”？这就是你刚才理解最神的地方：如果你想从第 6 行开始看，你的 **Offset** (偏离量) 必须是 5。因为：0(起点) + 5(偏离) = 5(索引)。而索引为 5 的那一行，在人类数数时正好是第 6 个。

## 东邪的 tip 1.4

三种注释—行内注释，整行注释 **/\* \*/**

## 定义 1.2 (排序)

**order by** 必须是最后的字句，

核心逻辑拆解

数据隔离：**SELECT** 决定了最后要把哪些“列”展示在屏幕上给用户看 [cite: 5, 2026-03-07]。

排序依据：**ORDER BY** 决定了数据库在后台检索数据时，用哪根“轴”来进行对比排序 [cite: 5, 2026-03-07]。

合法性：只要这一列存在于你 **FROM** 后面的那张表（或关联表）里，数据库就能抓到它来进行排序，即便你并没有要求显示它

 **笔记** 提示：通过非选择列进行排序

通常，**ORDER BY** 子句中使用的列将是为显示而选择的列。但是，实际上并不一定要这样，用非检索的列排序数据是完全合法的。

## 多列排序示例

```
/* 下面的代码检索 3 个列，并按其中两个列对结果进行排序——首先按价格，然后按名称排序。 */  
SELECT prod_id, prod_price, prod_name
```

```
FROM Products
ORDER BY prod_price, prod_name;
```

Listing 1.1: 使用列位置排序

```
SELECT prod_id, prod_price, prod_name
FROM Products
ORDER BY 2, 3;
```

这个就是先按照 price 排序再按照 name 排序也就是先价格再首字母。也就是按照 name 的首字母排序。用这个方法局限的是只能用 select 里面的列作为排序的依据。desc 可以按照降序排列他的作用只在他前面的东西。

**定义 1.3 (where 用来过滤数据)**

where 后面跟着限定条件比如 price=3.5; 记住 order by 得写在他的后面

**定义 1.4 (between)**

between 和 where 要一起用 where between 爱你的、

**定义 1.5 (or+and+In)**

where+and 可以进行高级筛选可以一个 where——多个 and, or 是或。and 的优先级比 or 高如果需要先进行 or 判断必须加括号。有 and 有 or 用括号。In (,) 需要有括号每两个不同的用, 隔开。也是 where in。not 否定跟着后面的条件在条件复杂的时候尤为有效

**东邪的 tip 1.5**

通配符用来匹配值一部分的特殊字符, 比如说含有 bean bag, like 得配合着 where 用, 立刻后面跟着 'fish'F

 **笔记** 下划线 (\_) vs 百分号 (%)

- % 是“贪婪”的: 它能匹配 0 个、1 个或无数个字符。比如 '% inch' 能匹配 8 inch, 也能匹配 12 inch。
- \_ 是“死板”的: 一个下划线就代表一个占位符。

看图 3 中的代码: LIKE '\_\_ inch teddy bear' (注意这里是两个下划线)。

 **作业**

1. 为什么 12 inch 和 18 inch 能匹配?  
因为 12 是两个字符, 刚好填满了那两个下划线的坑。
2. 为什么 8 inch 匹配不到?  
因为 8 只有一个字符。如果你用了两个下划线 \_\_, SQL 就会在心里数数: “一、二... 诶? 8 后面没字符了, 但我这里要求必须有两个字符才能过关”, 于是就把它剔除了。

**东邪的 tip 1.6 (and 后面还需要写一遍判断主体 prod\_descname)**

```
select prod_name, prod_desc from Products where prod_desc like "
```

## 1.1 计算字符

**输入**

```
SELECT prod_id,
       quantity,
       item_price,
       quantity*item_price AS expanded_price
```

```
FROM OrderItems
WHERE order_num = 20008;
```

输出

| prod_id | quantity | item_price | expanded_price |
|---------|----------|------------|----------------|
| RGAN01  | 5        | 4.9900     | 24.9500        |
| BR03    | 5        | 11.9900    | 59.9500        |
| BNBG01  | 10       | 3.4900     | 34.9000        |
| BNBG02  | 10       | 3.4900     | 34.9000        |
| BNBG03  | 10       | 3.4900     | 34.9000        |

#### 东邪的 tip 1.7

如何进行加减乘除，把运算完的起一个名字即可然后就可以搜索了

section 计算字符 计算字符涉及到了合并列等等操作要紧跟着 select

输入：

```
SELECT vend_name + ' (' + vend_country + ')'
FROM Vendors
ORDER BY vend_name;
```

输出：

|                 |            |
|-----------------|------------|
| Bear Emporium   | (USA )     |
| Bears R Us      | (USA )     |
| Doll House Inc. | (USA )     |
| Fun and Games   | (England ) |
| Furball Inc.    | (USA )     |
| Jouets et ours  | (France )  |

#### 东邪的 tip 1.8

+ 可以用来拼接'(' 是一个整体

输入

```
SELECT RTRIM(vend_name) + ' (' + RTRIM(vend_country) + ')'
FROM Vendors
ORDER BY vend_name;
```

输出

```
Bear Emporium (USA)
Bears R Us (USA)
Doll House Inc. (USA)
Fun and Games (England)
Furball Inc. (USA)
Jouets et ours (France)
```

#### 东邪的 tip 1.9

rtrim 函数可以去掉所有右边空格，比如这里去掉的是 `vend_nameRT + rimLfT`

输入



```
SELECT RTRIM(vend_name) + '_( ' + RTRIM(vend_country) + ' )'
       AS vend_title
FROM Vendors
ORDER BY vend_name;
```

**输出**

vend\_title

---

Bear Emporium (USA)  
 Bears R Us (USA)  
 Doll House Inc. (USA)  
 Fun and Games (England)  
 Furball Inc. (USA)  
 Jouets et ours (France)

**东邪的 tip 1.10**

as 赋予了新的列一个名字也就是合成的这个列也有名字了可以 select 了

## 1.2 函数

**东邪的 tip 1.11**

upper 函数会让所有字母都大写括号里面就是大写的对象

**表 1.1:** 常用的文本处理函数

| 函数                                  | 说明                  |
|-------------------------------------|---------------------|
| LEFT() (或使用子字符串函数)                  | 返回字符串左边的字符          |
| LENGTH() (或使用 DATALENGTH() 或 LEN()) | 返回字符串的长度            |
| LOWER()                             | 将字符串转换为小写           |
| LTRIM()                             | 去掉字符串左边的空格          |
| RIGHT() (或使用子字符串函数)                 | 返回字符串右边的字符          |
| RTRIM()                             | 去掉字符串右边的空格          |
| SUBSTR() 或 SUBSTRING()              | 提取字符串的组成部分 (见表 8-1) |
| SOUNDEX()                           | 返回字符串的 SOUNDEX 值    |
| UPPER()                             | 将字符串转换为大写           |

**东邪的 tip 1.12**

soundex() 方便寻找拼写错误的, 但是发生想的比如如果录入的时候 Michel 打错了大成 Michelle 了那么就可以在 where 后面加上判断条件 where soundex(vend\_name)=soundex(Michel)

字母拼起来不能用——要用 concat

**东邪的 tip 1.13**

```
select cust_id,cust_name, upper(CONCAT (left (cust_contact,2),left(cust_city,3))) as
user_logn from Customers
```

```
SELECT AVG(prod_price) AS avg_price
FROM Products;
```

输出：

```
avg_price
-----
6.823333
```

#### 东邪的 tip 1.14

记住每一次运算完都要给新结果一个名字。avg 只能运算单列会忽略 null 值，

- 使用 COUNT(\*) 对表中行的数目进行计数，不管表列中包含的是空值 (NULL) 还是非空值。
- 使用 COUNT(column) 对特定列中具有值的行进行计数，忽略 NULL 值。

#### 东邪的 tip 1.15

group by 是和 where 配合操作目的是对一类目标形成筛选和操作。group by 在 where 后面在 order by 前面

#### 笔记 GROUP BY 的使用注意事项：

- GROUP BY 子句中列出的每一项都必须是检索列或有效的表达式，但不能是聚集函数。也就是说，GROUP BY 后面应当写“具体的分组依据”，而不能写 COUNT(\*)、SUM(price) 这类分组之后才得到的结果。
- 如果在 SELECT 中使用了表达式，例如 YEAR(order\_date)，那么在 GROUP BY 中也必须写相同的表达式，而不能只按原列分组。
- 在很多 SQL 实现中，GROUP BY 中不能直接使用 SELECT 子句里定义的别名，因此通常应写原表达式，而不要写别名。
- 大多数 SQL 实现不允许对长度可变的大字段进行分组，例如文本型字段或备注型字段。

#### 笔记 WHERE 与 HAVING 的区别：

- WHERE 用于在分组之前对原始记录进行筛选，决定哪些行能够进入分组，因此 WHERE 是“管行”的。
- HAVING 用于在分组之后对分组结果进行筛选，决定哪些分组能够保留下来，因此 HAVING 是“管组”的。
- 一般来说，WHERE 中不能使用聚集函数，而 HAVING 中可以使用聚集函数。

**例题 1.1** 下面的 SQL 语句先筛选工资大于 5000 的员工，再按部门编号分组，最后只保留人数不少于 3 的部门：

```
SELECT dept_id, COUNT(*)
FROM employee
WHERE salary > 5000
GROUP BY dept_id
HAVING COUNT(*) >= 3;
```

#### 笔记 上述语句的执行过程可以理解为：

- 先由 WHERE salary > 5000 筛选出工资大于 5000 的员工记录；
- 再按照 dept\_id 对这些记录进行分组；
- 最后由 HAVING COUNT(\*) >= 3 保留员工人数不少于 3 的那些部门分组。

#### 东邪的 tip 1.16

聚集函数如果和 GROUP BY 一起出现，默认就是“对每个组聚集”。所以聚类函数 (aggregate function) count all 默认就是 count 刚分好的组。

 **笔记** 虽然 SELECT 子句写在最前面，但 SQL 查询在逻辑上并不是先执行 SELECT。一般可理解为按如下顺序执行：

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY.

因此，COUNT(\*) 虽然写在 SELECT 中，但如果语句里有 GROUP BY，它实际统计的是每个分组中的行数。

**表 1.2:** ORDER BY 与 GROUP BY 的区别

| ORDER BY               | GROUP BY                      |
|------------------------|-------------------------------|
| 对产生的输出排序               | 对行分组，但输出可能不是分组的顺序             |
| 任意列都可以使用（甚至非选择的列也可以使用） | 只可能使用选择列或表达式列，而且必须使用每个选择列表表达式 |
| 不一定需要                  | 如果与聚集函数一起使用列（或表达式），则必须使用      |

#### 东邪的 tip 1.17

用 group by 不能指望他能排好

**表 1.3:** ORDER BY 与 GROUP BY 的区别

| ORDER BY               | GROUP BY                      |
|------------------------|-------------------------------|
| 对产生的输出排序               | 对行分组，但输出可能不是分组的顺序             |
| 任意列都可以使用（甚至非选择的列也可以使用） | 只可能使用选择列或表达式列，而且必须使用每个选择列表表达式 |
| 不一定需要                  | 如果与聚集函数一起使用列（或表达式），则必须使用      |

**例题 1.2** `select order_num, count(*) as group_line from OrderItems group by order_num group by count`

**例题 1.3** 下面是一条 SQL 查询语句，用于查询订购了产品 RGAN01 的顾客编号：

```
SELECT cust_id
FROM Orders
WHERE order_num IN (SELECT order_num
                    FROM OrderItems
                    WHERE prod_id = 'RGAN01');
```

其输出结果为：

```
cust_id
-----
1000000004
1000000005
```

#### 东邪的 tip 1.18

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY 这里是从两个不同的表里面 select 并且把第一个的结果当成

**例题 1.4** 下面这个写法是一个典型错误：

```
SELECT cust_email
```



```

FROM Customers
WHERE cust_id IN (
    SELECT order_date, cust_id
    FROM Orders
    WHERE order_num IN (
        SELECT order_num
        FROM OrderItems
        WHERE prod_id = 'BR01'
    )
);

```

错误原因在于：外层条件是

`cust_id IN (子查询)`

这里左边只有一个值 `cust_id`，因此括号中的子查询必须只返回一列，并且这一列应当能够与 `cust_id` 进行比较。

但是该子查询返回了两列：

`order_date, cust_id`

这就与 `IN` 的要求不匹配，因此会报错。

正确写法应为：

```

SELECT cust_email
FROM Customers
WHERE cust_id IN (
    SELECT cust_id
    FROM Orders
    WHERE order_num IN (
        SELECT order_num
        FROM OrderItems
        WHERE prod_id = 'BR01'
    )
);

```

也就是说，当写成

某列 `IN` (子查询)

时，子查询通常只能返回一列；如果返回多列，就会出现这种典型错误。



**笔记** SQL 里子查询常见有三种位置：1. 写在 `WHERE` 后面：拿来筛选 2. 写在 `FROM` 后面：拿来当一张临时表 3. 写在 `SELECT` 后面：拿来给每一行补一个值 `select` 后面可以是表中没有的也就是处理过的新添加的

**例题 1.5** 下面的查询用于返回每个顾客的顾客 ID 以及其已订购商品的总金额，并按总金额从大到小排序：

```

SELECT cust_id,
    (SELECT SUM(quantity * item_price)
     FROM OrderItems
     WHERE order_num IN (SELECT order_num
                        FROM Orders
                        WHERE Orders.cust_id = Customers.cust_id)) AS total_ordered
FROM Customers
ORDER BY total_ordered DESC;

```

**东邪的 tip 1.19**

先把最终要输出的 `cust_id` 和 `as total_orders` 写在 `select` 后，这里子句明显是作为计算字段，因为要添加表里面原来没有的东西。所以 `select` 肯定在 `from` 前面。一共需要两步，我们先算总价格，这个是容易想的；但是算总价格这步不能建立起 `orders` 和 `customers` 的相同 `id` 的映射，因为 `where in` 这种筛选只能里面有一个 `select`，所以还需要一个筛选的 `select`。最后有一个 `as` 的名字就行了，`sum` 的时候就不用有名字。

**东邪的 tip 1.20 (一个顾客可以下好几个订单，每一个订单可以买好几个东西)**

建立两个表的联系就是 `where A in (select A from ...)`，`A` 的位置只能是一个。这里如果要计算 `sum`，就需要知道 `sum` 的是那个名字的所有的数量 \* 价格，所以我们必须要筛选相同 `order_num` 的东西。写出标题话我们就可以知道，算订单总价紧跟着的必须是订单号，筛选订单号紧跟着的必须是顾客 `id`。可以从侧栏点当前输入，还可以检查这个表里面到底有没有。

## 1.3 联结表 13/22

**东邪的 tip 1.21 (意义)**

把信息分解成不同的表，通过共同的值关联。• 供应商表：

供应商表: `vend_id, vend_name`

商品表: `prod_id, vend_id, prod_name`

```
SELECT vend_name, prod_name, prod_price
FROM Vendors
INNER JOIN Products ON Vendors.vend_id = Products.vend_id;
```



**笔记** 此语句中的 `SELECT` 与前面的 `SELECT` 语句相同，但 `FROM` 子句不同。这里，两个表之间的关系是以 `INNER JOIN` 指定的部分 `FROM` 子句。在使用这种语法时，联结条件用特定的 `ON` 子句而不是 `WHERE` 子句给出。传递给 `ON` 的实际条件与传递给 `WHERE` 的相同。

**东邪的 tip 1.22**

`inner join A on B` 很清晰的会知道是 `A` 和 `B` 做联结 • `WHERE`：逻辑上像先全组合再筛 • `INNER JOIN`：逻辑上更像“按条件配对”

**例题 1.6** 在多表联结中，若某个字段名在多个表中都出现过，就需要写成“表名.字段名”的形式，以消除歧义。例如这里的 `order_num` 在 `Orders` 表和 `OrderItems` 表中都可能出现，因此应写成 `Orders.order_num`。

另外，若在 `SELECT` 中使用了聚合函数 `sum(...)`，那么凡是同时出现但没有被聚合的字段，都必须写在 `GROUP BY` 中。所以 `cust_name` 和 `Orders.order_num` 都要分组。

对应的 SQL 语句为：

```
select cust_name, Orders.order_num,
       sum(quantity*item_price) as ordertotal
from Customers
inner join Orders
on Customers.cust_id = Orders.cust_id
inner join OrderItems
on OrderItems.order_num = Orders.order_num
group by cust_name, Orders.order_num;
```

**定义 1.6 (等联结)**

若两个表进行联结时，联结条件是两个字段之间的相等关系，即使用等号 = 来建立匹配关系，则这种联结称为等联结。

例如：Customers.cust\_id = Orders.cust\_id 和 Orders.order\_num = OrderItems.order\_num 都是等联结条件。

**东邪的 tip 1.23**

有 group by 再想筛选 group by 也就是分组后产生的结果就要用 having, where 只能在 group by 前面筛选分组前的内容

**例题 1.7** 下面语句用于查询订单总额不少于 1000 的顾客编号和订单编号：

```
select Orders.cust_id, OrderItems.order_num
from Orders
inner join OrderItems
on Orders.order_num = OrderItems.order_num
group by Orders.cust_id, OrderItems.order_num
having sum(item_price * quantity) >= 1000;
```

常见错误如下：

1. 将 OrderItems 误写为 OrdersItems。
2. 将聚合条件写在 where 中，而不是写在 having 中。
3. select 中出现的非聚合列没有全部写入 group by。
4. 多表查询时，对重名字段没有写成 表名. 字段名，从而产生歧义。

```
SELECT cust_name, cust_contact
FROM Customers AS C, Orders AS O, OrderItems AS OI
WHERE C.cust_id = O.cust_id
      AND OI.order_num = O.order_num
      AND prod_id = 'RGAN01';
```

这样就可以缩短表头的书写。

 **笔记** [自联结] 可以，因为这里不是两个不同的 Customers 表，而是同一张表起了两个别名：

- Customers AS c1
- Customers AS c2

相当于把同一张表“当成两份来看”。

这样做是为了让表自己和自己比较，这种连接称为自联结（self join）。

这句查询语句的含义是：

- c2.cust\_contact = 'Jim Jones': 先找到联系人是 Jim Jones 的那一行；
- 再利用 c1.cust\_name = c2.cust\_name，找到与他姓名相同的其他顾客。

所以，c1 和 c2 只是同一张表在查询中扮演的两个不同角色，并不冲突。

 **笔记**

1. INNER JOIN 只保留能匹配上的行。
2. OUTER JOIN 除了匹配上的行，还会保留没匹配上的一边或两边。也就是会给没有的补一个 null，必须前面有 right 或者 left 说明他到底补那边。还有 full 就是两边都补  
customer left outer join 就是左侧也就 customer 全部填满 select 是倒数第二优先级运行只比 order by 高一级

## 1.4 union

### 东邪的 tip 1.24

union 可以连接两个 select 但是两个的 select 内容必须格式相同。union 自动删除重复的项。如果想要重复的项就 union all，如果排序 order by 必须在最后一个 select 的最后面

**例题 1.8** 在 SQL 中，UNION 要求两个 SELECT 的结果结构兼容，即列数相同，且对应位置的数据类型可以匹配；它并不要求两边查询的是同一种含义的数据。

例如下面是可以的：

```
SELECT prod_name
FROM Products
UNION
SELECT cust_name
FROM Customers;
```

因为两边都只返回 1 列，且通常都是字符型。这个的效果就是两个合成一列  
但下面不可以：

```
SELECT prod_name, prod_id
FROM Products
UNION
SELECT cust_name
FROM Customers;
```

因为左边返回 2 列，而右边只返回 1 列，列数不一致。

### 东邪的 tip 1.25

insert into 要插的表加 (表的表头) value () 如果没有的值要用 null 填充。insert select 用法自动从新表 select 东西合并到老表里面

creat 新表 select

### 东邪的 tip 1.26 (update)

要更新的表的名字，要更新的内容，还有用于更新具体地方的条件。update+set。update 可以用子查询

```
UPDATE 表名
SET 列名 = 新值
WHERE 条件;
```

### 东邪的 tip 1.27 (delete)

可以删除特定的行和删除所有的行，他是删除行删除不了列

```
DELETE FROM 表名
WHERE 条件;
```

1. 第一个问题是最开始的语句写法不对，SET 后面不能只写 UPPER(vend\_state)，而要写成

```
vend_state = UPPER(vend_state)
```

2. 第二个问题是没有写 WHERE 条件，在 MySQL 的安全更新模式下，直接更新整张表会被禁止。
3. 第三个问题是后来虽然加了 WHERE vend\_state IS NOT NULL，但 vend\_state 不是键列（key column），所以在安全模式下仍然不能执行。

**东邪的 tip 1.28**

create table 就是后面跟着表名字然后（表头 + 数据类型 + 能否是 null 值，可以用 default 规定默认值，drop table 就能删除整个表）

```
CREATE TABLE 表名
(
    列名1 数据类型 约束,
    列名2 数据类型 DEFAULT 默认值
);
```

```
DROP TABLE 表名;
```

```
ALTER TABLE 表名
ADD 列名 数据类型;
```

```
ALTER TABLE 表名
DROP COLUMN 列名;
```

**东邪的 tip 1.29**

所以 view 就相当于一个加工好的东西，如果需要建新的或者覆盖只能删除了在建

假设不想让别人看到 prod\_desc，可以建一个视图只暴露三列：

```
CREATE VIEW ProductsPublic AS
SELECT prod_id, prod_name, prod_price
FROM Products;
```

以后查数据只需：

```
SELECT * FROM ProductsPublic;
```

prod\_desc 对用户完全不可见。若需简化重复查询，例如常用“价格超过 10 的产品”，可建视图：

```
CREATE VIEW ExpensiveProducts AS
SELECT prod_name, prod_price
FROM Products
WHERE prod_price > 10;
```

以后直接查询该视图即可，无需每次重写 WHERE 条件：

```
SELECT * FROM ExpensiveProducts;
```

**例题 1.9 JOIN 视图** 创建仅含下过单顾客的视图：

```
CREATE VIEW CustomersWithOrders AS
SELECT Customers.*, Orders.order_num, Orders.order_date
FROM Customers
JOIN Orders ON Customers.cust_id = Orders.cust_id;
```

常见错误：JOIN 后漏写 ON；用 SELECT \* 导致 cust\_id 重复 (Error 1060)；重建视图前未执行 DROP VIEW (Error 1050)。注意 INNER JOIN 已自动过滤未下单顾客，无需额外 WHERE。

**例题 1.10 计算列视图** 创建含总价计算列的视图：

```
CREATE VIEW OrderItemsExpanded AS
SELECT order_num, prod_id, quantity, item_price,
       quantity*item_price AS expanded_price
FROM OrderItems;
```

常见错误：在视图定义中写 ORDER BY，MySQL 不保证其生效，应在查询视图时再加排序。视图可直接存储计算列，避免每次查询重复计算。

## 1.5 高级用法

### 定义 1.7 (存储过程)

存储过程 (Stored Procedure) 是一组预编译的 SQL 语句集合，存储在数据库中可反复调用，语法为：

```
CREATE PROCEDURE 过程名(参数)
BEGIN
    -- 过程体
END;

-- 调用
CALL 过程名(参数);
```

与函数的区别在于：存储过程无需返回值，不能嵌套在 SELECT 中使用。



### 定义 1.8 (事务处理)

事务处理 (Transaction) 是一组 SQL 操作的集合，要么全部执行成功提交，要么全部回滚撤销，保证数据的一致性。核心语句为：

```
START TRANSACTION; -- 开始事务
COMMIT;           -- 提交 (全部成功则执行)
ROLLBACK;         -- 回滚 (出错则撤销所有操作)
```

事务具有四个基本性质 (ACID)：原子性 (Atomicity)、一致性 (Consistency)、隔离性 (Isolation)、持久性 (Durability)。



### 定义 1.9 (游标)

游标 (Cursor) 是一种数据库对象，用于逐行处理查询结果集，语法为：

```
DECLARE 游标名 CURSOR FOR SELECT 语句;
OPEN 游标名;
FETCH 游标名 INTO 变量名;
CLOSE 游标名;
```

游标通常在存储过程中使用，适用于需要对结果集逐行操作的场景。





**定义 1.10 (约束)**

约束 (Constraint) 是对表中数据的限制规则，常见类型如下：

```
PRIMARY KEY,      -- 主键，唯一标识每一行
FOREIGN KEY,      -- 外键，保证引用完整性
UNIQUE,           -- 唯一约束，列值不可重复
NOT NULL,         -- 非空约束
CHECK (条件)      -- 检查约束，限制列的取值范围
```

**定义 1.11 (索引)**

索引 (Index) 是对表中一列或多列的值建立的排序结构，用于加快查询速度，语法为：

```
CREATE INDEX idx_name ON 表名 (列名);
DROP INDEX idx_name ON 表名;
```

索引可以提升查询效率，但会降低插入、更新、删除的速度，且占用额外存储空间。

**定义 1.12 (触发器)**

触发器 (Trigger) 是在特定事件 (INSERT、UPDATE、DELETE) 发生时自动执行的 SQL 语句，语法为：

```
CREATE TRIGGER 触发器名
AFTER INSERT ON 表名
FOR EACH ROW
BEGIN
    -- 触发时执行的操作
END;
```

触发器常用于自动记录日志、维护数据一致性等场景。



## 第2章 练习

### 2.1 基础

```
select tweet_id
from Tweets
where length(content) > 15;
```

数字字符串长度是 length

### 2.2 连接

```
SELECT customer_id, COUNT(*) AS count_no_trans
FROM Visits
WHERE visit_id NOT IN (
    SELECT visit_id FROM Transactions
)
GROUP BY customer_id
```

#### 东邪的 tip 2.1

where not in 其实就相当于减法也就是交集，不是减具体数字的都可以用 where not in。想不明白先列式子是谁减去谁

```
SELECT w1.id
FROM Weather w1
JOIN Weather w2
ON DATEDIFF(w1.recordDate, w2.recordDate) = 1
WHERE w1.temperature > w2.temperature;
```

#### 东邪的 tip 2.2 (日期题)

MySQL 中常用的日期函数有两个：

- DATEDIFF(日期 1, 日期 2): 返回日期 1 - 日期 2 的天数差
- DATE\_ADD(日期, INTERVAL n DAY): 在某日期上加  $n$  天

#### 东邪的 tip 2.3

遇到需要自己表里面数据互相操作最好先连成一张表，如果不需要所有都连接就用子查询生成表的几列当成一个表在链接。弄成两个表就好操作了

```
SELECT a1.machine_id, ROUND(AVG(a2.timestamp - a1.timestamp), 3) AS processing_time
FROM Activity AS a1
JOIN Activity AS a2
ON a1.machine_id = a2.machine_id
AND a1.process_id = a2.process_id
AND a1.activity_type <> a2.activity_type
```

```
WHERE a2.activity_type = 'end'
GROUP BY a1.machine_id;
```

### 定义 2.1 (交叉连接 (CROSS JOIN))

交叉连接返回两张表的笛卡尔积，即左表的每一行与右表的每一行两两组合，结果行数为左表行数乘以右表行数。语法为：

```
SELECT * FROM 表A CROSS JOIN 表B;
```

例如 Students (4 行) 与 Subjects (3 门课) 做交叉连接，得到  $4 \times 3 = 12$  行，即每个学生与每门课的全部组合。常与 LEFT JOIN 配合使用，用于生成“所有可能组合”后再统计实际数据。



```
SELECT s.student_id, s.student_name, sub.subject_name,
       COUNT(e.student_id) AS attended_exams
FROM Students s
CROSS JOIN Subjects sub
LEFT JOIN Examinations e
ON s.student_id = e.student_id
AND sub.subject_name = e.subject_name
GROUP BY s.student_id, s.student_name, sub.subject_name
ORDER BY s.student_id, sub.subject_name;
```

### 东邪的 tip 2.4

SELECT 里凡是没有被聚合函数 (COUNT、SUM、AVG、MAX、MIN) 包裹的列，都必须出现在 GROUP BY 里。因为是要 select 了我就必须知道他要在哪个组里面

```
SELECT t.name FROM (
  SELECT e1.name, COUNT(e2.id) AS sum
  FROM Employee AS e1
  JOIN Employee AS e2 ON e1.id = e2.managerId
  GROUP BY e1.id, e1.name -- {原写法: GROUP BY managerId, e1.name}
) AS t
WHERE t.sum >= 5;
```

### 东邪的 tip 2.5

写 sql 的时候先把最后一步的上一步的表一个长什么样生成对了再进行下一步筛选，如果上一步还复杂就上上步直到出现简单的操作了；HAVING 是专门配合 GROUP BY 用的，用来过滤分组后的聚合结果。规则是：WHERE 过滤的是分组前的原始行 HAVING 过滤的是分组后的聚合值

```
SELECT e1.name
FROM Employee AS e1
JOIN Employee AS e2 ON e1.id = e2.managerId
GROUP BY e1.id, e1.name
HAVING COUNT(e2.id) >= 5;
```

## 2.3 函数

### 东邪的 tip 2.6

这题我的想法是自己需要行与行之间做差所以先自链接，链接需要 end 对 start 然后其他的 id 需要一致。然后需要做差也就是 end-start 也就是 a2-a1 需要 a2 对 type 是 end 最后 groupby machine\_id

```
SELECT machine_id,
       ROUND(AVG(CASE WHEN activity_type = 'end'
                       THEN timestamp
                       ELSE -timestamp END), 3) AS processing_time
FROM Activity
GROUP BY machine_id;
```



**笔记** CASE WHEN 是 SQL 中的条件表达式，相当于编程语言中的 if-else，可用于 SELECT、WHERE、ORDER BY 等子句中。语法为：

```
CASE WHEN 条件1 THEN 值1
      WHEN 条件2 THEN 值2
      ELSE 值3
END
```

例如在聚合函数中，将 end 的时间戳取正、start 的时间戳取负，直接用 AVG 即可得到平均运行时间：

```
AVG(CASE WHEN activity_type = 'end'
          THEN timestamp
          ELSE -timestamp END)
```

### 东邪的 tip 2.7

sql 里面的 if else 语句

```
SELECT action,
       CASE WHEN action = 'confirmed' THEN 1 ELSE 0 END AS is_confirmed
FROM Confirmations;
```

### 东邪的 tip 2.8

action 那行出来就是状态比如说 time out 或者 confirmed。然后这个新的列叫 is confirmed 里面就全是 0 和 1。然后 avg 就是求平均数，round (avg, 3) 就是保留三位小数的平均数。和计算函数一样一般放 select，后面。有 case 就必须有 end

```
SELECT s.user_id, IFNULL(stat, 0) AS confirmation_rate
FROM Signups AS s
LEFT JOIN (
    SELECT c.user_id, ROUND(AVG(CASE WHEN action = 'confirmed'
                                    THEN 1 ELSE 0 END), 2) AS stat
    FROM Confirmations AS c
    GROUP BY c.user_id
) AS cs ON s.user_id = cs.user_id;
```

**东邪的 tip 2.9**

先写子查询表，在用 join 把 signup 表和子查询表连起来，要注意的是如果要连必须给新的表起一个名字比如 cs，然后 ifnull(stat,0) 就是给 stat 位置是 null 的自动填写是 0

## 2.4 聚合

```
SELECT ps.product_id,
       IFNULL(ROUND(SUM(ps.price * us.units) / SUM(us.units), 2), 0) AS average_price
FROM UnitsSold AS us
RIGHT JOIN (SELECT * FROM Prices) AS ps
ON us.product_id = ps.product_id
AND us.purchase_date BETWEEN ps.start_date AND ps.end_date
GROUP BY ps.product_id;
```



**笔记** 相较于初始写法，本题做了三处修改：

- JOIN 改为 RIGHT JOIN：以 Prices 为右表，保留所有产品，即使没有销售记录也会出现在结果中
- 加上 IFNULL(..., 0)：RIGHT JOIN 后没有匹配的产品 average\_price 为 NULL，用 IFNULL 补 0
- GROUP BY us.product\_id 改为 GROUP BY ps.product\_id：右表没匹配到的行 us.product\_id 为 NULL，必须按 ps.product\_id 分组才能正确输出所有产品

**东邪的 tip 2.10**

select 内容一变化 group by 就必须变

**例题 2.1** 查询每个项目员工的平均经验年数：

```
SELECT p.project_id, ROUND(AVG(e.experience_years), 2) AS average_years
FROM Project AS p
LEFT JOIN Employee AS e ON p.employee_id = e.employee_id
GROUP BY p.project_id;
```

**第一步：LEFT JOIN**，按 employee\_id 连接两表，每行变成“项目 id + 员工经验年数”：

| project_id | experience_years |
|------------|------------------|
| 1          | 3                |
| 1          | 2                |
| 1          | 1                |
| 2          | 3                |
| 2          | 2                |

**第二步：GROUP BY project\_id**，按项目分组：

project\_id 1 → [3, 2, 1], project\_id 2 → [3, 2]

**第三步：AVG**，对每组求平均：

project\_id 1 →  $\frac{3+2+1}{3} = 2.00$ , project\_id 2 →  $\frac{3+2}{2} = 2.50$

**第四步：ROUND**，保留两位小数输出。

**东邪的 tip 2.11**

group by 和计算的可以不是一个

 **笔记** [主要是主键有问题] 本题的三处易错点：第一，ROUND 的位数参数写成 0 而非 2，导致结果无小数；第二，子查询统计总用户数时用了 COUNT(DISTINCT user\_name)，应为 COUNT(user\_id)，user\_name 不是主键，用错列会导致分母计算错误；第三，Register 表的主键是 (contest\_id, user\_id) 组合，本身不会重复，COUNT 时无需加 DISTINCT。

### 东邪的 tip 2.12

主键不能是 null 不能重复可以唯一识别

 **笔记** MySQL 中，条件表达式（比如 rating < 3）直接放在数值计算里，会自动返回 1（真）或 0（假）。  
例如：

```
SELECT rating < 3 FROM Queries;
```

结果就是一列 0 和 1。

所以 AVG(rating < 3) 等价于：

```
SUM(CASE WHEN rating < 3 THEN 1 ELSE 0 END) / COUNT(*)
```

这是 MySQL 特有的行为，标准 SQL 和其他数据库（如 PostgreSQL、SQL Server）不支持，要用 CASE WHEN 才行。力扣默认 MySQL 所以能用，但面试手写 SQL 最好还是写 CASE WHEN，更通用。

### 定义 2.2 (DATE\_FORMAT 函数)

DATE\_FORMAT(date, format) 将日期按指定格式转换为字符串，常用格式符如下：

- %Y：四位年份，如 2019
- %m：两位月份，如 01
- %d：两位日期，如 07

例如提取年月：

```
SELECT DATE_FORMAT(trans_date, '%Y-%m') AS month FROM Transactions;
```

结果将 2019-01-07 转换为 2019-01，常用于按月分组：

```
GROUP BY DATE_FORMAT(trans_date, '%Y-%m')
```



```
SELECT DATE_FORMAT(trans_date, '%Y-%m') AS month,
       country,
       COUNT(state) AS trans_count,
       SUM(amount) AS trans_total_amount,
       SUM(state = 'approved') AS approved_count,
       SUM(CASE WHEN state = 'approved' THEN amount ELSE 0 END) AS approved_total_amount
FROM Transactions
GROUP BY country, month;
```

### 东邪的 tip 2.13

sum(state='approved') 自动能给 state 赋值布尔值是 1，不是赋 0，然后就可以 sum。然后 case 后面要跟着 when，then 后面也可以写变量名

 **笔记** 两者本质不同：

- SUM(state='approved')：利用 MySQL 把布尔表达式自动转成 0/1，只能用于计数。



- SUM(CASE WHEN state='approved' THEN amount ELSE 0 END): THEN 后面跟的是真实字段值 amount, 可以是任意数值, 用于按条件求和。

所以不能混用——SUM(state='approved' \* amount) 这种写法是错的, 布尔值和字段值是两回事, 必须用 CASE WHEN 才能在条件成立时取字段值。

### 定义 2.3 (MIN / MAX 函数)

MIN 和 MAX 是聚合函数, 可作用于数值、字符串和日期类型。

- MIN(col): 返回该列的最小值, 用于日期时返回最早日期
- MAX(col): 返回该列的最大值, 用于日期时返回最晚日期

MySQL 中日期按 YYYY-MM-DD 格式存储, 从左到右依次比较年、月、日, 因此 MIN、MAX、>、< 对日期均可直接使用。例如:

```
SELECT customer_id, MIN(order_date) AS first_order_date
FROM Delivery
GROUP BY customer_id;
```

返回每位顾客最早的下单日期, 即首次订单日期。



 **笔记** WHERE order\_date IN (子查询) 中子查询返回的是每个顾客的最早日期, 但如果两个不同顾客恰好在同一天下了首次订单, IN 会把两个人的那天所有订单都选进来, 不只是首次订单那一行。更安全的写法是用 (customer\_id, order\_date) 组合匹配:

```
SELECT ROUND(AVG(order_date = customer_pref_delivery_date) * 100, 2)
      AS immediate_percentage
FROM Delivery
WHERE (customer_id, order_date) IN (
  SELECT customer_id, MIN(order_date)
  FROM Delivery
  GROUP BY customer_id
);
```

```
SELECT ROUND(AVG(order_date = customer_pref_delivery_date) * 100, 2)
      AS immediate_percentage
FROM Delivery
WHERE (customer_id, order_date) IN (
  SELECT customer_id, order_date
  FROM Delivery
  GROUP BY customer_id
  HAVING order_date = MIN(order_date)
);
```

### 东邪的 tip 2.14

having 还是不熟练

### 定义 2.4 (DATE\_ADD 与 INTERVAL)

DATE\_ADD(date, INTERVAL n unit) 在日期上加上指定时间间隔, 常用单位如下:

- DAY: 天, 如加一天
- MONTH: 月, 如加一个月

- YEAR: 年, 如加一年



**例题 2.2** 例如在日期上加一天:

```
SELECT DATE_ADD('2016-03-01', INTERVAL 1 DAY);
-- 结果: 2016-03-02
```

也可以用减法:

```
SELECT DATE_ADD('2016-03-01', INTERVAL -1 DAY);
-- 结果: 2016-02-29
```

常用于判断某日期的前后一天是否存在记录:

```
WHERE event_date = DATE_ADD(first_login, INTERVAL 1 DAY)
```

### 东邪的 tip 2.15

- 函数名写错了, 是 DATE\_ADD 不是 add\_date。
- DATE\_ADD(firstdate, INTERVAL 1 DAY) 里的 firstdate 是刚起的别名, 但 MySQL 不允许在同一层 SELECT 里引用刚定义的别名, 需要嵌套一层子查询。

```
SELECT ROUND(
    COUNT(DISTINCT a1.player_id) / (SELECT COUNT(DISTINCT player_id) FROM Activity),
    2
) AS fraction
FROM Activity AS a1
JOIN (
    SELECT player_id, MIN(event_date) AS firstdate
    FROM Activity
    GROUP BY player_id
) AS t ON a1.player_id = t.player_id
AND a1.event_date = DATE_ADD(t.firstdate, INTERVAL 1 DAY);
```

### 东邪的 tip 2.16

今天的错误是太贪心, 想少用子查询。首先应该想明白到底要连接几张表。是这样的, 比如最后满足我们要求的只有一个元素, 这个需要 COUNT 没问题, 但是一旦进行操作以后最开始的表一定变化了, 所以一定需要一个最开始表的子查询, 也就是 (SELECT COUNT(DISTINCT player\_id) FROM Activity)。除数是一个表, 被除数是一个表, 求除数再看需要几个表。一个表没法和自己的元素上下判断, 所以需要自连接弄成横着的。然后筛选条件一定要写在一起 DATE\_ADD(t.firstdate, INTERVAL 1 DAY), 最后再筛选。



**笔记** GROUP BY product\_id 之后每组变成一行, 但 sale\_date 这一组里有多值, MySQL 不知道该显示哪一个, 所以不允许直接 SELECT。GROUP BY 之后 SELECT 的列只能是 GROUP BY 里出现的列或聚合函数 (MIN、MAX、COUNT、SUM、AVG)。

```
SELECT product_id,
    COUNT(*) AS 销售次数,
    MIN(sale_date) AS 最早销售,
    MAX(sale_date) AS 最晚销售
FROM Sales
```

```
GROUP BY product_id;
```

 **笔记** WHERE 后面不能带聚合函数，但 HAVING 可以，因为执行顺序不同：

FROM → WHERE → GROUP BY → HAVING → SELECT

WHERE 在 GROUP BY 之前执行，此时还没有分组，聚合函数根本没算出来，所以不能用。HAVING 在 GROUP BY 之后执行，聚合函数已经算好了，所以可以用。

 **笔记** 聚合函数的作用域是“组内”，不是“全表”

一旦有 GROUP BY，所有聚合函数（MAX、SUM、COUNT 等）只在当前组的行上计算，不跨组。

易错点：GROUP BY num 之后每组已折叠为一行，组内 num 全部相同，此时 MAX(num) 等于原值，毫无意义。

正确做法：内层分组过滤，外层跨组聚合。

```
-- 错误：MAX 只在组内算，等于原值
SELECT MAX(num)
FROM MyNumbers
GROUP BY num
HAVING COUNT(*) < 2;

-- 正确：子查询先得到候选值，外层再取最大
SELECT MAX(num)
FROM (
    SELECT num
    FROM MyNumbers
    GROUP BY num
    HAVING COUNT(*) < 2
) t;
```

## 2.5 高级查询和链接

 **笔记** CASE WHEN 是 SQL 里的条件判断，相当于 if-else，可放在以下位置：

- SELECT 后面（最常用，生成新列）
- WHERE 后面（条件筛选）
- ORDER BY 后面（按条件排序）
- GROUP BY 后面（按条件分组）

```
CASE
    WHEN 条件1 THEN 结果1
    WHEN 条件2 THEN 结果2
    ELSE 默认结果
END
```

 **笔记** 三角形判断条件等价于最大边小于另外两边之和：

$$\text{GREATEST}(x, y, z) < x + y + z - \text{GREATEST}(x, y, z)$$

```
CASE
    WHEN GREATEST(x, y, z) < x + y + z - GREATEST(x, y, z)
    THEN 'Yes'
    ELSE 'No'
```

```
END AS triangle
```

 **笔记** MAX/MIN 是聚合函数，作用于多行之间的比较（跨行），不能用于同一行内多列的比较。GREATEST/LEAST 是标量函数，作用于同一行内多个值之间的比较（跨列）。

## 2.5.1 窗口函数

### 定义 2.5 (窗口函数)

窗口函数在保留原表所有行的基础上，对每一行计算其“窗口”（周围若干行）内的聚合或排序结果，不像 GROUP BY 那样折叠行。基本语法为：

```
函数名() OVER (
  PARTITION BY 分组列 -- 可省略
  ORDER BY 排序列
)
```

常用窗口函数：

- LAG(col, n)：当前行往上第  $n$  行的值
- LEAD(col, n)：当前行往下第  $n$  行的值
- RANK()：当前行的排名（并列跳号）
- ROW\_NUMBER()：当前行的行号（不跳号）



 **笔记** OVER (ORDER BY ...) 中的排序仅作用于窗口函数的计算过程，相当于一张临时工作视图，不影响最终结果的行顺序。执行流程为：

原表（无序）

- > OVER (ORDER BY score DESC) -- 生成临时排序视图，供窗口函数计算
- > 窗口函数在临时视图上计算结果
- > 结果贴回原表行，按原表顺序输出

### 东邪的 tip 2.17

窗口函数是自连接的高级替代品，覆盖了自连接的常见用途，同时能表达自连接表达不了的“相邻行”语义，性能也更好。

 **笔记** OVER (ORDER BY ...) 后可跟多列，逗号分隔，规则与普通 ORDER BY 相同。常用窗口函数语义如下：

- RANK()：按排序结果给排名，并列则同名次，下一名跳号
- ROW\_NUMBER()：按排序结果给行号，并列也强制不同，严格连续
- LAG(col, n)：当前行在排序视图中，往上第  $n$  行的 col 值；第一行往上无数据则为 NULL
- LEAD(col, n)：当前行在排序视图中，往下第  $n$  行的 col 值；最后一行往下无数据则为 NULL

### 东邪的 tip 2.18

where 在 select 前面所以不能用 select 新造的名字在 where 里面

```
select product_id,new_price from Products where( product_id,change_date) in(select product_id,max(
  change_date)as new
from Products where change_date < '2019-08-16' group by product_id);
--然后先把每一个查询需要提供什么想清楚把子查询就留在上面。然后复制来看有没有对齐什么的
select distinct p1.product_id,ifnull(p2.new_price,10)as price
```

```

from Products p1 left join(
select product_id,new_price from Products where( product_id,change_date) in(select product_id,max(
change_date)as new
from Products where change_date <= '2019-08-16' group by product_id))as p2
on p1.product_id=p2.product_id

```

**东邪的 tip 2.19**

IN 只匹配日期值，不匹配产品，所以如果两个都要匹配得弄成数组形式。把做好的子查询留在上面方便查看哪里有问题。



**笔记** 普通聚合函数加上 OVER 即变为窗口函数，不折叠行，每行保留并附上窗口内的聚合结果。

```

SUM() OVER ()
AVG() OVER ()
MAX() OVER ()
MIN() OVER ()
COUNT() OVER ()

```

例如 `SUM(weight)OVER (ORDER BY turn)` 表示按 turn 顺序累计求和，每行得到的是”到当前行为止所有人的体重总和”。

```

SELECT person_name
FROM (
SELECT person_name,
SUM(weight) OVER (ORDER BY turn) AS total
FROM Queue
) AS f
WHERE f.total <= 1000
ORDER BY total DESC
LIMIT 1;

```

但是为了追求运行效率还是 leftjoin 一个你需要所有元素的基础表比较好。

**定义 2.6 (LEFT JOIN 强制保留分类)**

当 GROUP BY 无法保证所有分类出现时，先用 UNION ALL 构造枚举模板表作为左表，再 LEFT JOIN 实际统计结果，缺失分类得到 NULL，用 COALESCE 转为 0。

```

SELECT category, COALESCE(accounts_count, 0) AS accounts_count
FROM (
SELECT 'Low_Salary' AS category UNION ALL
SELECT 'Average_Salary' AS category UNION ALL
SELECT 'High_Salary' AS category
) AS categories
LEFT JOIN (
SELECT
CASE
WHEN income < 20000 THEN 'Low_Salary'
WHEN income BETWEEN 20000 AND 50000 THEN 'Average_Salary'
ELSE 'High_Salary'
END AS category,

```

```

COUNT(*) AS accounts_count
FROM Accounts
GROUP BY category
) AS counts USING (category);

```



**解** 按新 *id* 升序返回每两个连续学生的交换结果。

```

SELECT
CASE
    WHEN id % 2 = 1 AND id = (SELECT MAX(id) FROM Seat) THEN id
    WHEN id % 2 = 1 THEN id + 1
    ELSE id - 1
END AS id,
student
FROM Seat
ORDER BY id;

```

 **笔记** CASE WHEN 按分支顺序逐一判断，匹配即停止，因此第一个条件必须是最特殊的情形。本题中须先排除“最后一个奇数行”这一特例，再处理一般奇数行；若将两个条件对调，最后一行会错误地执行 *id* + 1，导致结果越界。

#### 东邪的 tip 2.20

这个题的关键就是通过调整 *id* 来调整名单

### 2.5.2 union all

#### 定义 2.7 (UNION ALL)

将多个 SELECT 语句的结果集纵向拼接，要求各子查询的列数与对应列的数据类型一致。

```

SELECT col1, col2 FROM table1
UNION ALL
SELECT col1, col2 FROM table2;

```

UNION ALL 保留所有行（含重复）；UNION 额外执行去重，性能较差。列名以第一个 SELECT 为准。



想把两个结果搜好了再竖着拼起来最后输出

```

SELECT name AS results
FROM (
    结果1
) AS t1
UNION ALL
SELECT title AS results
FROM (
    结果2
) AS t2;

```

## 2.6 子查询

子查询就是创建一个池子以前要写 a 在 1, 2, 3 里面选但是用另一个表表里就是 1, 2, 3。就用 select from 这个表直接返回 1, 2, 3。当然作为比较条件也可以是大余小余找第二大的去掉最大的就行。

```
SELECT MAX(Salary) AS SecondHighestSalary
FROM Employee
WHERE Salary < (SELECT MAX(Salary) FROM Employee);
```

### 定义 2.8 (SQL 自定义函数)

自定义函数 (User-Defined Function) 是用户在数据库中创建的可复用逻辑单元，语法为：

```
CREATE FUNCTION 函数名(参数名 参数类型, ...) RETURNS 返回类型
BEGIN
    -- 函数体
    RETURN 返回值;
END
```

与存储过程的区别在于：函数必须有返回值，且可以直接嵌套在 SELECT 语句中使用。

### 定义 2.9 (SQL 循环)

循环只能写在存储过程或自定义函数内部，MySQL 提供三种循环结构：

```
-- 1. LOOP: 手动控制退出
loop1: LOOP
    IF 条件 THEN LEAVE loop1; END IF;
END LOOP;

-- 2. WHILE: 先判断条件再执行
WHILE 条件 DO
    -- 执行内容
END WHILE;

-- 3. REPEAT: 先执行再判断，至少执行一次
REPEAT
    -- 执行内容
UNTIL 条件 END REPEAT;
```

### 例题 2.3 \*[177 第 n 高薪水]

```
CREATE FUNCTION getNthHighestSalary(N INT) RETURNS INT
BEGIN
    SET N = N - 1;
    RETURN (
        SELECT DISTINCT Salary
        FROM Employee
        ORDER BY Salary DESC
        LIMIT 1 OFFSET N
    );
END
```



